CS 432 Distributed Systems	Lab (4)

Section Content:

- **Naming**

What is a Name?

A **name** is a string of characters (letters, digits, symbols) used to **refer to an entity**.

An **entity** can be almost anything in a system: a file, a device, a server, a user, a service, a web page, or a printer.

Example: report.pdf is the name of a file, **www.google.com** is the name of a website, and **printer-floor2** is the name of a printer.

What is Naming?

Naming is the **whole process and system** we use to give names to entities and later find them using those names. It has two parts:

1. **Binding:** associating a name with an entity (giving it a name).
2. **Name Resolution:** when someone asks for a name, the system figures out what it refers to and where it is.

Name ⟶ (Name Resolution) ⟶ Address ⟶ Actual Entity
"www.google.com" 142.250.x.x The server

Simplify: Think of your phone's contact list. You save "Mom" (the name) next to her number (the address). When you tap "Mom", your phone translates the name into a number and dials it. That is naming in action.

The component that does this job is called a **Naming System** or **Name Service**. A famous example is **DNS** (Domain Name System).

Why Do We Need Naming in a Distributed System?

A distributed system has hundreds or thousands of machines and services spread across a network. Without names, it would be chaos.

Reason	Explanation
Resource Sharing	For one machine to use a file or service on another machine, it must be able to refer to it.
Human-friendliness	People remember google.com far more easily than 142.250.185.78.
Location Transparency	Users should not need to know where a service physically runs.
Mobility / Migration	If we move a service to another server, the name stays the same and only the mapping changes behind the scenes.
Replication / Load Balancing	One name can lead to many replica servers providing the same service.
Scalability	As the system grows, we need an organized way to find anything quickly.

Example: If your website moves to a new server with a different IP, users still type the same name, and only the DNS record is updated. Without naming, every user would need to learn the new IP.

Difference Between Name, Address, and Identifier

Name

- Refers to a resource in a **human-friendly** way.
- Does **not** have to be unique, so different things can share the same name.
- Can change.

Address

- Tells you **where** the resource is and how to reach it.
- **Changes** if the resource moves (it is location-dependent).

Identifier

- Refers to a resource **uniquely** and permanently.
- It has three properties:
  1. It refers to **at most one entity**.
  2. Each entity is referred to by **at most one identifier**.
  3. It is **never reused** for another entity, even if the original is deleted.
- Usually **not human-friendly** (numbers, UUIDs).

Comparison

	Name	Address	Identifier
Purpose	Names the resource	Says where it is	Uniquely distinguishes it
Unique?	Not necessarily	Unique to one location	Always
Can change?	Possibly	Yes, if the resource moves	No, it is permanent
Human-friendly?	Yes	Usually not	Usually not
Location-dependent?	No	Yes	No

Example 1: The Citizen

Element	Value	Why?
Name	Mohamed Ahmed	Easy to remember, but 10 students could share it
Address	Apt 5, Tahrir Street, Cairo	Tells you where to find him, and changes if he moves
Identifier	National ID Number	Unique, permanent, never given to anyone else

Ambiguity in Naming

A name is **ambiguous** when it does not point to exactly one entity. In other words, when someone uses the name, the system (or the person) can't be sure which entity is meant.

Simple example: Two students in the class are both called "Mohamed Ahmed". If the teacher says "Mohamed Ahmed, please come up", who should stand? That is ambiguity.

The same name refers to different things.

Name	Could mean
server1	server1 in the Cairo data center, or server1 in the London data center
index.html	the file in project A, or the file in project B
Mohamed Ahmed	Student #1, Student #2, Student #3

How Do We Solve Ambiguity?

Solution	How it helps	Example
Hierarchical names (namespaces)	Add a path or domain so the name is unique within its parent	cairo.company.com/server1 vs london.company.com/server1
Fully qualified names	Write the complete name, not a shortened one	www.example.com. instead of www
Identifiers (UUIDs / IDs)	Use a unique, permanent value that never repeats	National ID Number

Naming Types

In a distributed system, there are **three types** of names. Each one is designed for a different purpose.

1. Human-Readable Names

Names designed for **people** to read, type, and remember.

- Made of meaningful words.
- Easy to remember, but **not necessarily unique**.
- Can be changed without changing the entity.
- A computer must **resolve** them before using them.

Example	Refers to
www.google.com	A website
report_final.pdf	A file
printer-floor2	A printer
/home/Users/notes.txt	A file path

Pros: easy for humans. **Cons:** can be ambiguous.

2. Addresses

The name of an **access point**, which tells you **where** to reach the entity.

- Used by machines to actually **locate and connect**.
- **Location-dependent:** changes if the entity moves.
- Not friendly for humans.

Example	Type
142.250.185.78	IPv4 address
192.168.1.10:8080	IP + port (a service endpoint)

Pros: directly usable by the network. **Cons:** breaks when the entity moves.

3. Identifiers

A name that identifies an entity **uniquely and permanently**.

- Refers to **one entity only**, and each entity has **one identifier only**.
- **Never reused**, even if the entity is deleted.
- Usually not human-friendly.

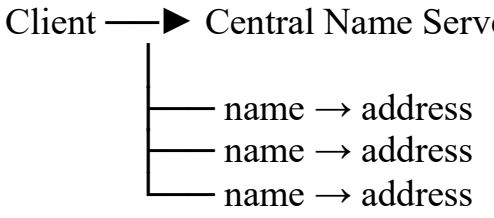
Example	Refers to
550e8400-e29b-41d4-a716-446655440000	A UUID for a service or record
National ID number	One person

Pros: never ambiguous, stable. **Cons:** hard to remember, needs a mapping to be useful to humans.

Naming Implementation Approaches

Centralized Naming

One server stores everything:



Simple, but if the server fails, the naming service may stop.

Also, if everyone asks this server, it becomes a **bottleneck**.

Distributed Naming

Instead of one server, we use multiple servers:

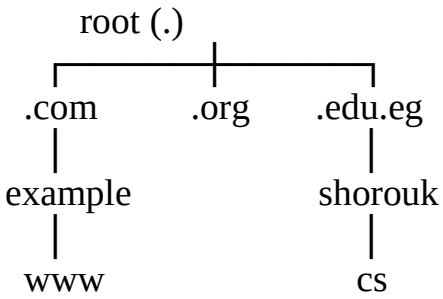


The information is distributed between them.

This is more **scalable** and avoids depending on one server, but it is more complicated.

Hierarchical Naming

Names are organized as a **tree**, and each part of the tree is managed by a different authority. This is the most common way to distribute naming.



- A name is written as a path through the tree: cs.shorouk.edu.eg
- Each level only needs to be unique **within its parent**, so www.example.com and www.shorouk.edu.eg don't clash.
- Each part of the tree can be **delegated** to a different owner (this solves the ambiguity problem).

Name Resolution

Name resolution is the process of taking a name and finding the **entity or address** it is bound to.

Two ways to resolve a name

a) **Iterative resolution:** the client does the work. Each server answers "I don't know, but ask this server next."

Client ⟶ Server 1 : "Where is cs.shorouk.edu.eg?"
Client ⟵ Server 1 : "Ask Server 2"
Client ⟶ Server 2 : "Where is cs.shorouk.edu.eg?"
Client ⟵ Server 2 : "Ask Server 3"
Client ⟶ Server 3 : "Where is cs.shorouk.edu.eg?"
Client ⟵ Server 3 : "It's 10.0.0.5"

b) **Recursive resolution:** the server does the work. The client asks once, and the server asks the others on its behalf.

Client ⟶ Server 1 ⟶ Server 2 ⟶ Server 3
Client ⟵ Server 1 ⟵ Server 2 ⟵ Server 3 (answer: 10.0.0.5)

DNS (Domain Name System) as a Real Example

These three alternatives show **different ways a client can find a host** when the local DNS server doesn't already know the answer. The first two are variations of **recursive** resolution, and the third is **iterative**.

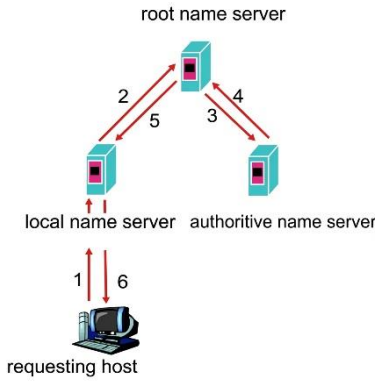
Key words

- Local DNS→The first server you ask
- Root→Tells you where to start looking
- Intermediate→Takes you closer to the responsible server
- Authoritative→Has the official final answer

1st Alternative: Local → Root → Authoritative (Recursive, 2 levels)

The simplest case: the root server **directly knows** the authoritative server.

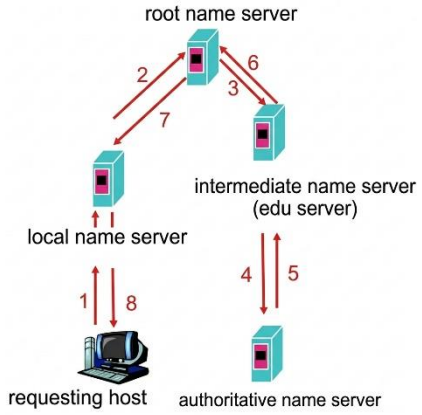
Client ⟶ Local DNS ⟶ Root Server ⟶ Authoritative Server
Client ⟵ Local DNS ⟵ Root Server ⟵ Authoritative Server (returns the host)



2nd Alternative: Local → Root → Intermediate → Authoritative (Recursive, 3 levels)

More realistic: the root **does not know** the authoritative server, but it knows an **intermediate** server that does.

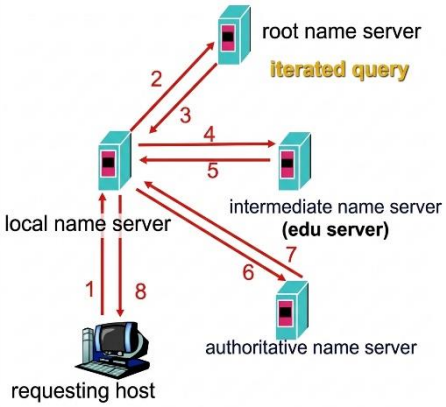
Client ⟶ Local DNS ⟶ Root ⟶ Intermediate ⟶ Authoritative
Client ⟵ Local DNS ⟵ Root ⟵ Intermediate ⟵ Authoritative (returns the host)



3rd Alternative: Iterative Queries

Here the servers **don't do the chasing**. Each server only replies with **"I don't know, but ask this server next."** The **local name server** does all the work.

Local DNS ⟶ Root Server : "Where is www.cs.shorouk.edu.eg?"
Local DNS ⟵ Root Server : "Don't know, ask the .eg server"
Local DNS ⟶ Intermediate Server : "Where is www.cs.shorouk.edu.eg?"
Local DNS ⟵ Intermediate Server : "Don't know, ask the shorouk.edu.eg server"
Local DNS ⟶ Authoritative Server : "Where is www.cs.shorouk.edu.eg?"
Local DNS ⟵ Authoritative Server : "It's 10.0.0.5"



Location Transparency

Location transparency means the user can access a resource **without knowing where it is physically or which server hosts it.**

The user uses a **name**, and the naming service finds the current address.
User → Name → Naming Service → Address → Resource

Example:

portal.shorouk.edu.eg → 10.0.0.5

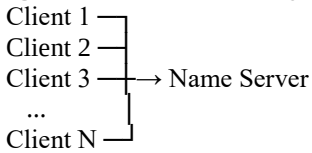
If the portal moves:

portal.shorouk.edu.eg → 10.0.8.20

The **name stays the same**; only the **name → address binding** changes.
Why? Because the name does not contain the physical location.
So users do not need to change bookmarks or applications when the resource moves.

Concurrency in Naming Services

A naming service is used by **many clients at the same time.**



Concurrent Reads

Many clients can ask for names simultaneously.
Reads are generally safe because they **do not modify the data**.

Problems are mainly:

- Overload
- Bottlenecks

Solutions:

- **Replication** → multiple servers share the requests.
- **Caching** → store answers temporarily near clients.
- **Load balancing** → distribute requests.
- **Multithreading** → handle multiple requests concurrently.

Programs

Program1:

Below is a simple program that simulates a Naming Service (Node Registry) in a distributed system. The program creates several nodes, stores them in an ArrayList, and displays information about all registered nodes.

Code

```
package noderegistry;
import java.util.ArrayList;
class Node {
    String id;
    String address;
    // Constructor
    Node(String id, String address) {
       this.id = id;
       this.address = address;
    }
    public String Display() {
       return "Node ID: " + id + ", Address: " + address;
    }
}
public class NodeRegistry {
    public static void main(String[] args) {
        // Create a list to store nodes
        ArrayList<Node> registry = new ArrayList<>();
        // Add nodes to the registry
        registry.add(new Node("N1", " 2.168.1.10"));
        registry.add(new Node("N2", " 2.168.1.11"));
        registry.add(new Node("N3", " 2.168.1.12"));
         // Display all registered nodes
        System.out.println("Registered Nodes:");
        for (Node node : registry) {
           System.out.println(node.Display());
        }
    }

}
```

Output

```
Registered Nodes:
Node ID: N1, Address:  2.168.1.10
Node ID: N2, Address:  2.168.1.11
Node ID: N3, Address:  2.168.1.12
BUILD SUCCESSFUL (total time: 0 seconds)
```

Program2:

Here is a simple program that demonstrates Location Transparency using a Naming Service (Registry). The client uses a logical name ("printer") instead of knowing the actual IP address and port number.

Code

```
package transparencydemo;
import java.util.HashMap;
import java.util.Map;
class NamingService {
    // Registry that maps logical names to physical addresses
    private static Map<String, String> registry = new HashMap<>();
    static {
        registry.put("printer", " 2.168.1.50:9100");
        registry.put("database", " 2.168.1.60:   ");
    }
    // Resolve a logical name into a physical address
    public static String resolve(String logicalName) {
          return registry.get(logicalName);
    }
}
public class TransparencyDemo {
   public static void main(String[] args) {
        // Client only knows the logical name
        String target = NamingService.resolve("printer");
        System.out.println("Sending print job to: " + target);
        // The client never needs to know the real IP or port
   }

}
```

Output

```
Sending print job to:  2.168.1.50:9100
BUILD SUCCESSFUL (total time: 0 seconds)
```